Predicting College Football Win Probability Using Engineered Play-by-Play Features

Alvin Hampton

INFO 516 • Spring 2026

Introduction

College football generates an enormous amount of structured data. Every snap produces a record of the game state — who has the ball, where they are on the field, how much time is left — and over eight seasons that adds up to over 1.7 million individual play events. The question this project tries to answer is straightforward: given everything we know about the game state right now, what is the probability that the team with possession ultimately wins? Answering it rigorously is not.

The challenge is that win probability estimation is fundamentally different from standard outcome prediction. A model that picks the right winner 83% of the time is not necessarily useful if its probability estimates are unstable — jumping from 80% to 40% on a single incompletion and back to 70% on the next play. What broadcasters and analysts actually need is a model whose probability estimates are trustworthy at every moment of a game, not just correct on average. That distinction between accuracy and calibration drove every decision in this pipeline.

This project builds a complete win probability engine from data collection through deployment. Eight seasons of FBS play-by-play data (2017–2025, excluding 2020) were processed through a four-stage cleaning and feature engineering pipeline, resulting in a 24-feature dataset used to train two production models: a regularized logistic regression and an XGBoost gradient boosted classifier. Both are deployed in a Streamlit application that lets users replay any game play-by-play with live win probability updates. The most interesting finding is that the linear model and the tree-based model end up performing almost identically — which, as discussed later, says more about the feature engineering than the models themselves.

Data Collection and Cleaning

Data was collected from collegefootballdata.com using the R package cfbfastR, which handles batch API calls across seasons and teams. The final dataset covers all FBS games from 2017 through 2025, minus 2020, and comes out to 1,718,250 rows and 362 columns. Every row is a play. The 2020 season was excluded because the COVID-shortened, irregular schedule would have introduced structural inconsistencies that are hard to control for.

The first thing that had to happen before any modeling was identifying and removing data leakage. Of those 362 columns, 114 contained post-snap information — yards gained, EPA from the current play, end-of-play field position — that would not be available in a live prediction scenario. Removing them was non-negotiable. After that, the remaining features still had

substantial missing data despite what the provider advertised. Most numeric fields were imputed using within-game, team-level means, which at least keeps the context local. Anything with more than 15% missing was dropped. The target variable — whether the possession team ultimately won — was never imputed; any row missing it was removed.

A few other structural problems needed surgical fixes. Overtime plays were filtered out entirely, since they operate under completely different rules and would corrupt the time-based urgency signals the model relies on. About 0.5% of games had duplicate or skipped play sequence numbers, which were resolved through secondary sorting. Some rows had score changes logged before the corresponding scoring play, so score differential was recalculated from the start-of-play score rather than the end.

Preprocessing and Feature Engineering

Getting from the raw data to a model-ready dataset required four distinct preprocessing steps, each addressing a specific problem in how the raw data represents the game.

The most important fix was the game clock. Raw data resets the clock at halftime — 1,800 seconds counts down to zero, then resets for the second half. This means 90 seconds remaining in the second quarter and 90 seconds remaining in the fourth quarter look identical to the model, even though those situations could not be more different strategically. The fix was to create a new variable, adj_TimeSecsRem, that runs continuously from 3,600 seconds at kickoff to zero at the final whistle. For second-half plays, the remaining time is added to the 1,800 seconds of the completed first half. It sounds simple, but this single change meaningfully improves how the model understands urgency.

The second fix addressed field position. In goal-to-go situations, the raw data frequently recorded the distance to a first down as ten yards even when the offense was on the two-yard line — a logical impossibility, since the first down marker would be behind the back of the end zone. This was corrupting the model's sense of red zone leverage. The fix was a conditional cap: whenever the goal-to-go flag is active, distance is forced to equal yards remaining to the goal line. Timeout data had a different problem — it was sparsely populated throughout the raw records. Because timeouts reset to three at the start of each half and can only go down from there, mean imputation would be wrong. Instead, a forward-fill was applied grouped by game and half, so the most recent known value carries forward until a new one is observed, resetting at the start of the third quarter.

The third step was perspective alignment. The raw data ties everything to the home team, which would force the model to learn team-specific patterns — essentially memorizing that certain programs tend to win rather than learning why any team with a late lead tends to win. To fix this, all features were reframed relative to the team currently with possession. ELO ratings became pos_team_elo and def_pos_team_elo; the Vegas spread became pos_team_spread, flipped when the away team has the ball. This way the model learns that being a seven-point favorite matters,

not that Alabama specifically tends to win. It also makes the model robust to neutral-site games and postseason matchups where home/away designations are arbitrary.

The fourth step was a technical one. Running window-based rolling calculations on 1.6 million rows in Pandas caused repeated out-of-memory failures. PySpark was brought in for the heavy lifting — chronological sorting within each game, grouped window functions for momentum features, and the lag(1) shifts needed to prevent leakage. Once all the feature engineering was done, the 362 columns were reduced to the 24 features that actually matter, and the result was converted back to a Pandas DataFrame for modeling.

Feature Construction

The 24 features in the production dataset were not selected from the raw columns — they were built. The goal was to give the model pre-digested signals that encode how football actually works, rather than making it figure that out on its own from raw inputs. Features fall into four categories:

- **Base Game State (11)**: down, distance, yards to goal, adjusted time remaining, score differential, timeouts for both teams, home team indicator, ELO ratings for both teams, and the possession-aligned Vegas spread.
- **Momentum (5)**: cumulative yards on the current drive, drive success rate, drive time elapsed, rolling EPA over the last five plays, and rolling yards over the last five plays.
- **Interaction Terms (5)**: inter_score_x_time (score differential × fraction of game elapsed), inter_spread_x_time (spread × fraction of time remaining), inter_down_x_distance, inter_elo_diff, and inter_timeout_diff.
- **Polynomial Terms (3)**: sign-preserved squared score differential, squared adjusted time remaining, and squared yards to goal.

The interaction and polynomial terms are where the real modeling logic lives. A seven-point lead in the first quarter and a seven-point lead with two minutes remaining are completely different situations, but raw score differential treats them the same. inter_score_x_time fixes this by multiplying score differential by the fraction of game time that has elapsed. At kickoff the value is zero regardless of score; late in the fourth quarter, a seven-point lead produces a value close to 6.77. The model can then learn to weight that signal appropriately based on context, not just magnitude. inter_spread_x_time does something similar for pre-game expectations — it lets the Vegas spread influence the model early in the game when there's no live data yet, and then gradually fades that influence as the actual game takes over. This prevents the model from stubbornly holding onto pre-game narratives when an upset is happening on the field.

The polynomial terms address a similar gap. A 14-point lead is not twice as safe as a 7-point lead — it is exponentially safer because the trailing team needs to score twice without a response. A linear score differential feature cannot capture that. The sign-preserved squared score differential

encodes the non-linear relationship between lead size and win probability without requiring a tree model to discover it through splits.

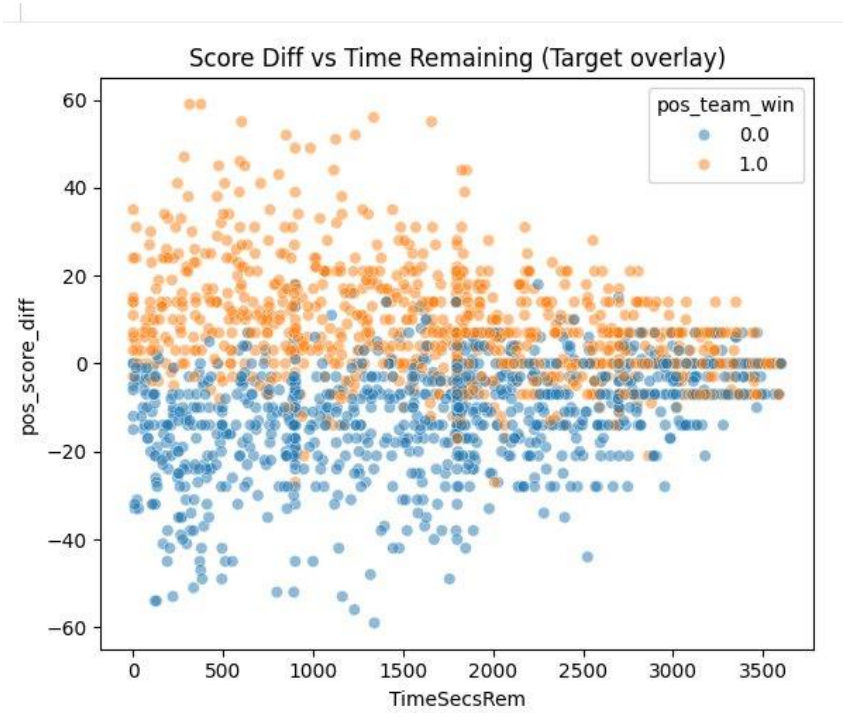

Figure 1: *Score differential versus adjusted time remaining, color-coded by win outcome (orange = possession team wins, blue = possession team loses). Early in games (right side), the same score differentials produce mixed outcomes. Near the final whistle (left side), the separation is clean — providing direct visual motivation for inter_score_x_time.*

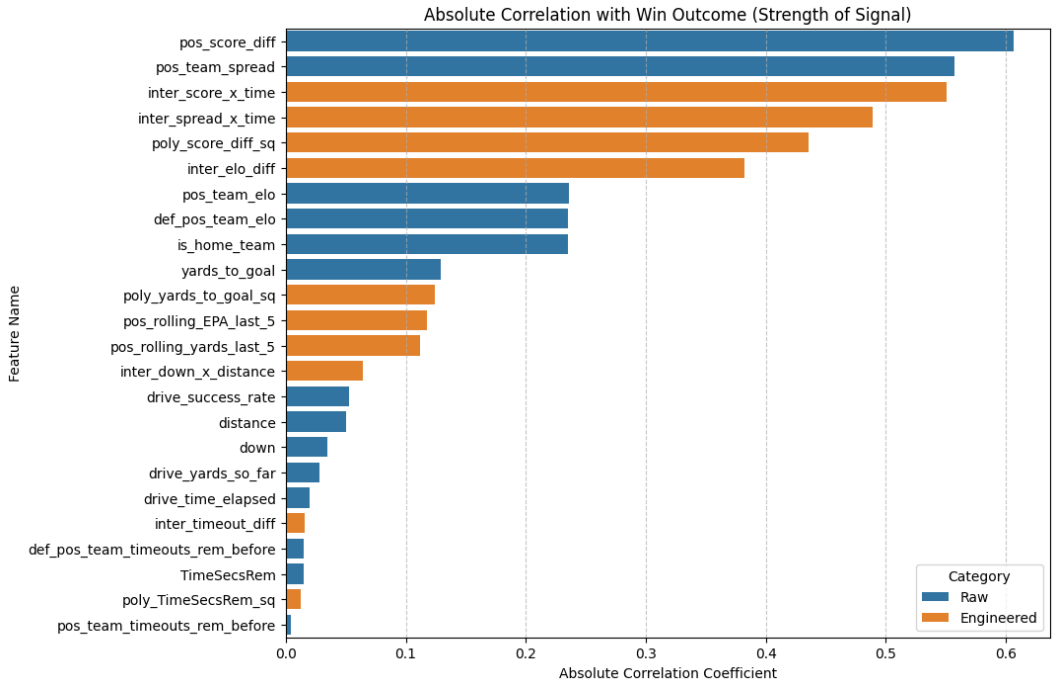

Figure 2: *Absolute correlation of each feature with the win outcome, by category. Engineered features (orange) consistently outrank the raw inputs they were derived from, accounting for approximately 60% of total predictive signal despite representing only 54% of the feature columns.*

Statistical Validation

Before relying on the models for anything meaningful, it made sense to verify that the engineered features actually carry real statistical signal and are not just artifacts of the construction process. A full logistic regression was run using statsmodels on the complete 1,590,878-play training sample. This gives coefficient estimates, standard errors, z-scores, and p-values for every feature — the kind of inferential output that scikit-learn does not produce. The model converged cleanly and achieved a Pseudo R^2 of 0.501, meaning the 24 features explain roughly half of the maximum possible improvement in log-likelihood over a null model. For play-by-play sports data, that is a strong result.

The headline finding was that inter_score_x_time returned a z-score of 129.3 with $p = 0.000$. That is not a rounding artifact — the feature is genuinely and overwhelmingly significant. The core game state and engineered features followed suit: pos_score_diff ($z = 71.6$), inter_spread_x_time ($z = -85.4$), pos_team_spread ($z = -69.3$), poly_yards_to_goal_sq ($z = -14.3$), and poly_score_diff_sq ($z = -11.6$) were all highly significant at $p < 0.001$. The momentum features also held up — drive success rate, rolling EPA, and rolling yards were all significant.

A few features came back with NaN standard errors and undefined p-values, which initially looks alarming but has a clean explanation. The three ELO-related features — pos_team_elo, def_pos_team_elo, and inter_elo_diff — are in perfect multicollinear relationship: inter_elo_diff is literally the difference of the other two. The design matrix is singular along that dimension, so the maximum likelihood estimator cannot separate the three coefficients. The L2 penalty in the production scikit-learn model handles this by shrinking all three proportionally rather than trying to isolate them. The timeout features showed a related problem — standard errors around 33,000 and p-values of 1.000, which is the classic sign of near-perfect collinearity between the individual timeout counts and their difference.

The non-significant features like poly_TimeSecsRem_sq ($p = 0.218$) and inter_down_x_distance ($p = 0.592$) are a more interesting case. These are not failing because they lack football signal. They are failing because that signal is already captured more efficiently by the dominant interaction terms. If you removed inter_score_x_time from the model, the p-values for several of these features would sharpen dramatically. Their statistical redundancy is actually evidence that the feature engineering worked — the constructed signals absorbed their contribution.

```
Iterations 10
                           Logit Regression Results
==============================================================================
Dep. Variable:           pos_team_win   No. Observations:              1590878
Model:                          Logit   Df Residuals:                  1590855
Method:                           MLE   Df Model:                           22
Date:                Mon, 27 Apr 2026   Pseudo R-squ.:                  0.5010
Time:                        17:56:50   Log-Likelihood:            -5.5027e+05
converged:                       True   LL-Null:                   -1.1026e+06
Covariance Type:            nonrobust   LLR p-value:                     0.000
=====================================================================================================
                                        coef    std err          z      P>|z|      [0.025      0.975]
-----------------------------------------------------------------------------------------------------
const                                -0.0331      0.003    -12.659      0.000      -0.038      -0.028
down                                 -0.1030      0.006    -16.376      0.000      -0.115      -0.091
distance                             -0.0334      0.007     -4.767      0.000      -0.047      -0.020
yards_to_goal                        -0.0191      0.011     -1.787      0.074      -0.040       0.002
TimeSecsRem                          -0.0516      0.010     -5.329      0.000      -0.071      -0.033
pos_score_diff                        1.1463      0.016     71.610      0.000       1.115       1.178
pos_team_timeouts_rem_before          0.0077   3.35e+04   2.29e-07      1.000   -6.57e+04    6.57e+04
def_pos_team_timeouts_rem_before      0.0069   3.32e+04   2.07e-07      1.000    -6.5e+04     6.5e+04
pos_team_spread                      -0.6742      0.010    -69.330      0.000      -0.693      -0.655
pos_team_elo                          0.0207        nan        nan        nan         nan         nan
def_pos_team_elo                     -0.0143        nan        nan        nan         nan         nan
is_home_team                          0.0641      0.003     25.062      0.000       0.059       0.069
drive_time_elapsed                   -0.0268      0.002    -11.046      0.000      -0.032      -0.022
drive_yards_so_far                   -0.0383      0.004     -8.774      0.000      -0.047      -0.030
drive_success_rate                    0.0360      0.004     10.114      0.000       0.029       0.043
pos_rolling_EPA_last_5                0.0065      0.003      2.217      0.027       0.001       0.012
pos_rolling_yards_last_5              0.0126      0.003      4.403      0.000       0.007       0.018
inter_score_x_time                    2.1484      0.017    129.325      0.000       2.116       2.181
inter_down_x_distance                -0.0042      0.008     -0.536      0.592      -0.020       0.011
inter_elo_diff                        0.0284        nan        nan        nan         nan         nan
inter_timeout_diff                    0.0007   4.09e+04   1.75e-08      1.000   -8.02e+04    8.02e+04
inter_spread_x_time                  -0.7047      0.008    -85.363      0.000      -0.721      -0.688
poly_TimeSecsRem_sq                  -0.0113      0.009     -1.232      0.218      -0.029       0.007
poly_yards_to_goal_sq                -0.1447      0.010    -14.349      0.000      -0.165      -0.125
poly_score_diff_sq                   -0.3505      0.030    -11.570      0.000      -0.410      -0.291
=====================================================================================================
```

Figure 3: *Full statsmodels Logit output for the 24-feature model on 1,590,878 training plays (Pseudo R^2 = 0.501). Features with NaN standard errors reflect perfect multicollinearity by construction. Timeout features (p = 1.000, std err ≈33,000) reflect near-perfect collinearity with inter_timeout_diff. Both are resolved by L2 regularization in the production models.*

Model Training and Performance

Two models were trained: a regularized logistic regression and an XGBoost gradient boosted classifier. Before fitting either, all features were standardized using StandardScaler — this is required for L2 regularization to penalize coefficients fairly, since features on different scales would otherwise receive disproportionate penalties. The train/test split used GroupShuffleSplit on game_id rather than a random split. That distinction matters: a random split would let the model train on early plays from a game and then evaluate on later plays from the same game, which is a subtle but real form of leakage. Grouping by game means every play in evaluation comes from a game the model has never seen.

The logistic regression used an L2 penalty with the SAGA solver, which converges efficiently on datasets at this scale. The optimal regularization strength of C = 0.1 was identified through GridSearchCV with three-fold cross-validation. That relatively strong penalty was necessary because the engineered features, by design, are correlated with each other — interaction terms are functions of base variables, and polynomials share variance with their linear counterparts. Without regularization, the optimizer would assign explosive opposing weights to correlated

pairs, producing coefficients that are large and unstable. The L2 penalty distributes the weight smoothly across correlated features instead, which is exactly what you want for a model that needs to produce smooth probability curves rather than erratic jumps.

XGBoost was configured with 500 estimators, a learning rate of 0.01, and a maximum depth of 3. The shallow depth was a deliberate choice. Deep trees would overfit to game-specific noise and produce steppy, discontinuous probability transitions that look wrong in the application — a single two-yard gain at midfield should not cause a five-percent probability swing. Keeping depth at three forces the model to focus on globally dominant signals and produces smooth, gradual probability movements. The conservative learning rate compounds this effect, ensuring the ensemble builds up slowly and generalizes better to seasons outside the training data. After hyperparameter selection through RandomizedSearchCV, both models were retrained on the full eight-season dataset so every available play contributed to the production models.

Metric	Logistic Regression	XGBoost
Log Loss	0.3437	0.3488
Brier Score	0.1121	0.1113
ROC-AUC	0.9245	0.9263
Accuracy	83.34%	83.51%

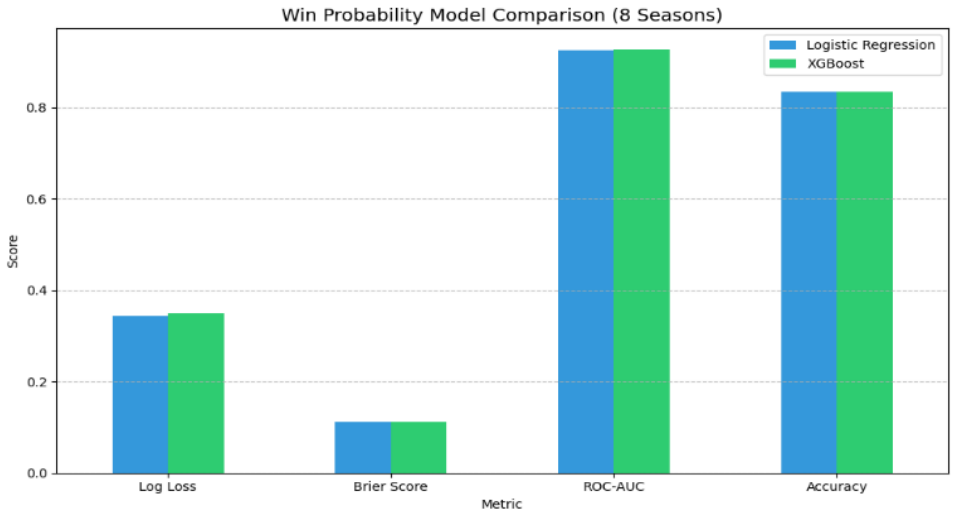

Figure 4: *Side-by-side metric comparison for both production models on held-out test data. The near-identical performance across all four metrics is the central finding of this analysis.*

The results were remarkably similar between the two models. Logistic regression achieved an AUC of 0.9245, a Brier score of 0.1121, a log loss of 0.3437, and 83.34% accuracy. XGBoost came in at 0.9263, 0.1113, 0.3488, and 83.51%. Any of those differences are well within noise.

A Brier score below 0.13 indicates that the model's probability estimates are genuinely calibrated — when it says 70%, teams actually win around 70% of the time. An AUC above 0.92 means that in 92% of random pairings between a winning play and a losing play, the model assigns a higher probability to the winning one.

Model Analysis

Looking at what each model actually learned is more illuminating than the aggregate metrics. In the logistic regression, the standardized coefficient for inter_score_x_time was approximately 2.14 — more than twice the magnitude of the next-largest feature. This confirms that the model learned exactly what the feature was designed to encode: scoring leads become dramatically more predictive as the game progresses. The pre-game spread and its time-decayed interaction ranked second and third, which makes intuitive sense — early in games there is minimal live data, so pre-game expectations carry more weight. Several features, including the timeout interaction and down-distance interaction, were suppressed near zero by the regularization penalty. That is not a failure; it means their predictive signal was already captured by more dominant terms.

XGBoost told a similar story through a different lens. Feature importance by gain showed poly_score_diff_sq accounting for 41.9% of total predictive weight, followed by pos_score_diff at 31.8%, inter_spread_x_time at 8.5%, inter_score_x_time at 5.1%, and pos_team_spread at 3.9%. The dominance of the squared score differential term is particularly interesting — it means XGBoost independently confirmed through its splitting process what the polynomial feature was designed to capture: that large leads are non-linearly safer than small ones. It found this even though the raw score differential was already in the feature set.

The multicollinearity between engineered features and their base variables is worth acknowledging directly. As Figure 5 shows, inter_score_x_time correlates at 0.95 with pos_score_diff, and poly_TimeSecsRem_sq correlates at 0.97 with TimeSecsRem. These relationships are built in by construction. For logistic regression, the L2 penalty prevents this from destabilizing coefficient estimates. For XGBoost, tree splitting naturally handles it — at any given node, the model picks the most informative split and ignores redundant alternatives. In both cases, the correlated features collectively represent the game state more richly than any individual feature could.

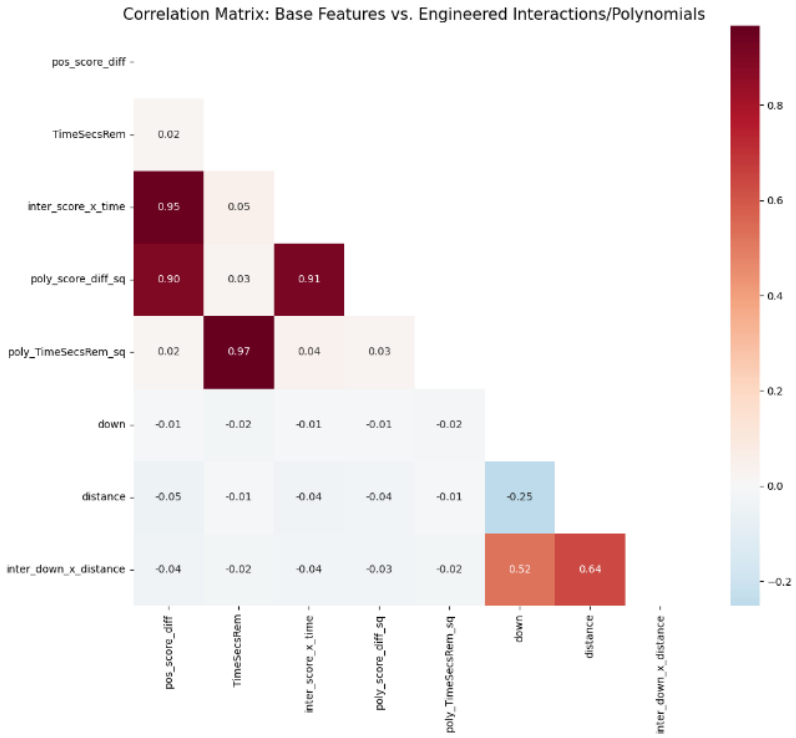

Figure 5: *Correlation matrix between base features and their engineered counterparts. High correlations are expected and by design; they reflect the mathematical relationships between raw variables and the interaction and polynomial terms derived from them.*

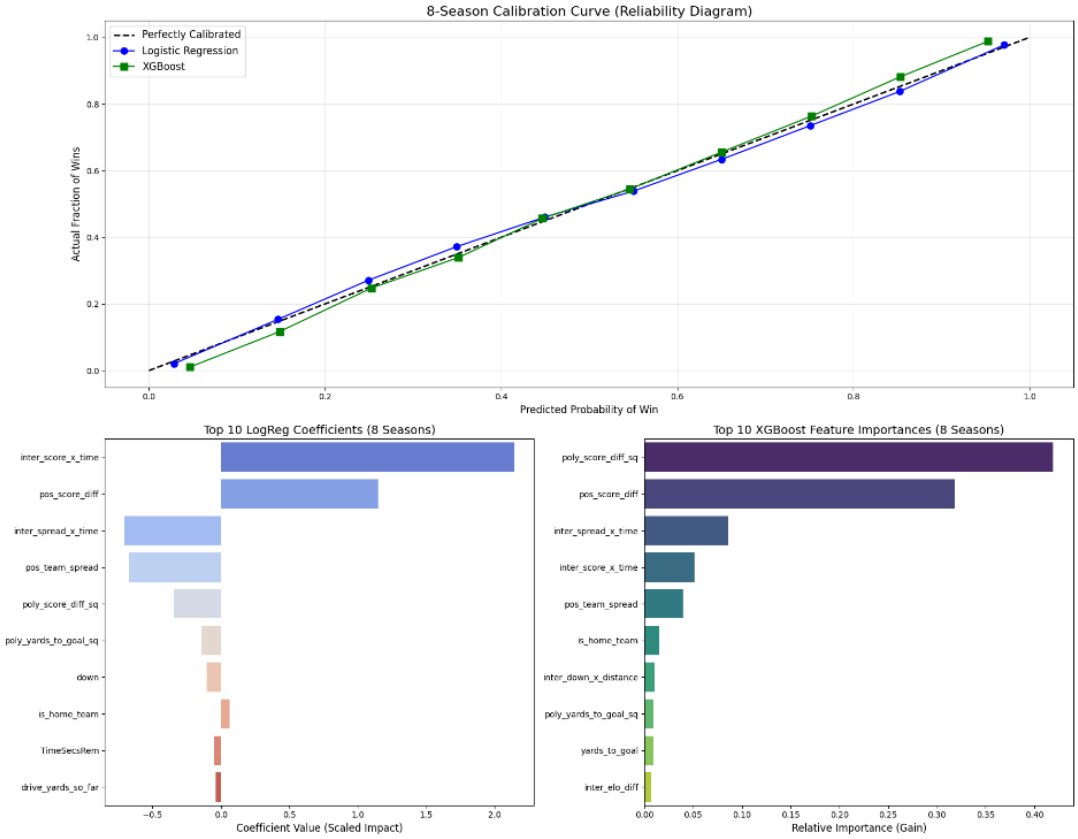

Figure 6: *Production model evaluation. Top: calibration curves for both models against the perfectly calibrated baseline — both track closely across the full probability range. Bottom-left: Logistic Regression coefficients, with inter_score_x_time dominating. Bottom-right: XGBoost feature importance by gain, with poly_score_diff_sq accounting for the largest share.*

The reason both models converge to essentially the same performance is not a coincidence — it is the point. When a regularized linear model matches a 500-tree gradient boosted ensemble on a 1.6 million play dataset, it means the feature engineering successfully linearized the non-linear structure of the problem. By constructing interaction terms and polynomial features that encode the physics of football directly, the pipeline gave the logistic regression exactly what it needed to compete with a model that discovers non-linear patterns automatically. Engineered features represent 54% of columns but carry approximately 60% of total predictive signal, an efficiency ratio of 1.38 for logistic regression and 1.26 for XGBoost. Feature quality beat model complexity.

Interactive Application

Both models are deployed in a Streamlit application that functions as a live win probability tracker. Users can select any game from the dataset, play through it snap by snap or jump to any point, and watch the win probability update in real time. The application was built around three core design decisions that are worth explaining.

The first is what might be called the perspective-flip. The model is trained to predict possession-team win probability, which is the right framing for learning football mechanics — positive field position and momentum always map to positive probability regardless of which team has the ball. But displaying raw model output directly would create a jarring visualization: on every turnover, the probability would jump discontinuously (from, say, 80% to 20%) as the possession team switches. To fix this, the application anchors all output to the home team. When the home team has the ball, the raw model output is shown. When the away team has the ball, the complement (1 minus the model output) is shown instead. The result is a single continuous line across the entire game, where turnovers appear as directional movements rather than vertical discontinuities — which is exactly how broadcast win probability graphics look.

The second is the Play of the Game feature. After computing win probability for every play, the app calculates the change in probability between consecutive plays from the winning team's perspective. The play with the largest positive swing is automatically flagged. In a game decided by a late interception or a goal-line stand, this surfaces the right moment without any manual inspection of the play log.

The third is performance. Both models and the fitted StandardScaler are loaded into memory at startup using Streamlit's cache_resource decorator, which keeps them resident for the full session. This eliminates the per-request disk overhead that would otherwise make interactive use impractical. The application processes all plays in a typical 200-play game and renders the complete win probability chart in under 1.5 seconds.

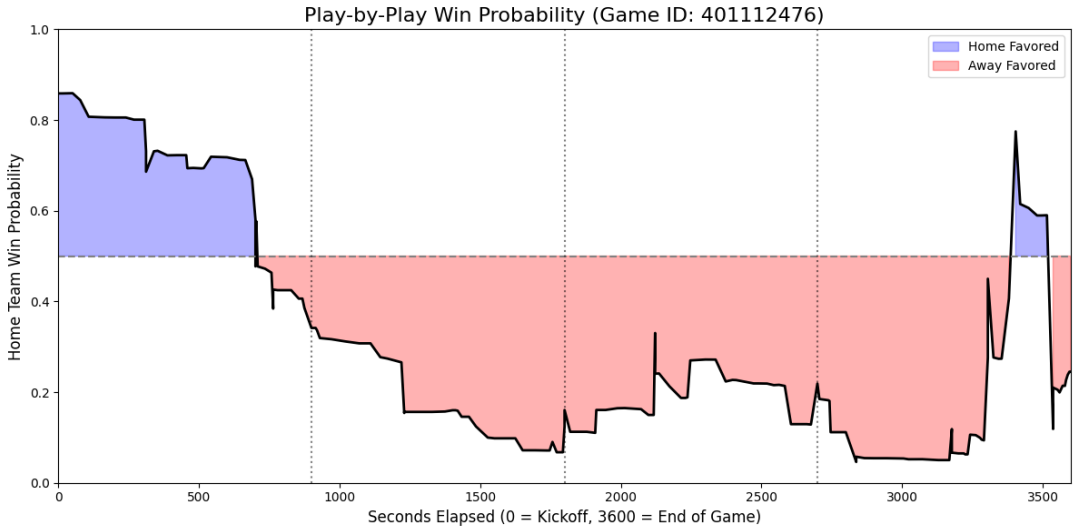

Figure 7: *Sample win probability chart for a single game, anchored to the home team throughout. Turnovers appear as smooth directional movements rather than vertical jumps, and the largest probability swing is automatically identified as the Play of the Game.*

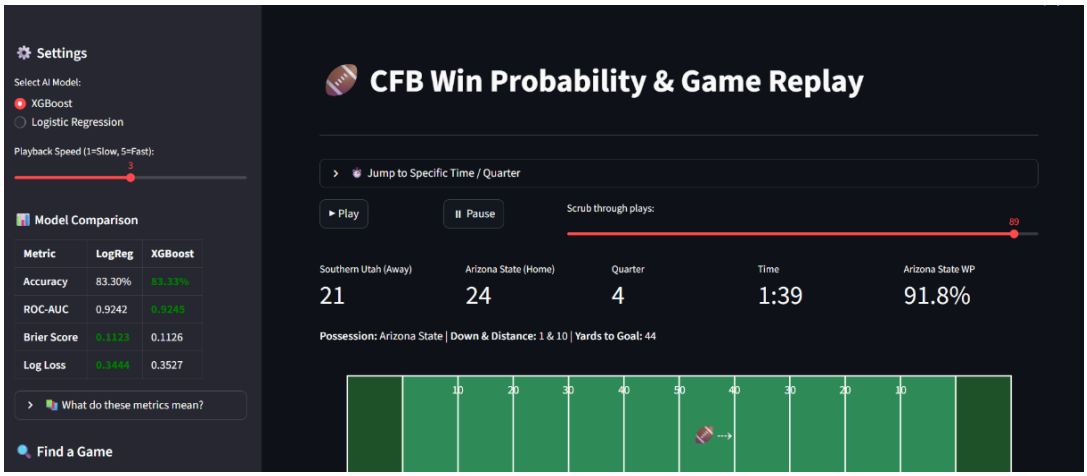

Figure 8: *The Streamlit application interface. The sidebar shows model selection and live metric comparison between XGBoost and Logistic Regression. The main panel displays current game state and win probability — in this example, Arizona State at 91.8% with 1:39 remaining in the fourth quarter.*

Conclusion

The central finding of this project is that a well-engineered feature set matters more than model architecture. A regularized logistic regression and a 500-tree XGBoost ensemble trained on the same 24 features produce nearly identical results across every evaluation metric: ROC-AUC of 0.924–0.926, accuracy around 83.4%, Brier scores below 0.12, and log loss around 0.34–0.35. That convergence is not a limitation — it is the payoff from spending time on the feature engineering rather than just trying bigger models.

The statsmodels validation added another layer of confidence. Pseudo R^2 of 0.501, a z-score of 129.3 for the primary interaction feature, and near-universal significance across the core engineered terms confirm that the features carry real predictive signal. The non-significant and undefined results for ELO and timeout features are mathematically explainable consequences of the construction strategy, not signs of a poorly designed feature set. The redundancy paradox — where engineered features render their base variables statistically redundant — is perhaps the cleanest evidence that the approach worked.

There are obvious directions to extend this work. Player-level features like quarterback EPA under pressure or defensive grades by unit could capture personnel-driven momentum that a play-level model misses entirely. A recurrent architecture like an LSTM trained on the sequence of play embeddings could model longer-range game flow dynamics more naturally than a five-play rolling window. And expanding to earlier seasons or improving coverage of non-Power Five programs would help the model generalize more broadly. For now, the current system produces calibrated, broadcast-quality win probability estimates across the full spectrum of FBS football, and does it fast enough to be genuinely useful in a live application context.